ML Interview Preparation Notes

LLM Inference, GPU Systems, and Production Serving

ML Interviews Q/A Project

Updated 25 October 2026

Contents

1 LLM inference and serving **2**
- 1.1 Q1. What happens between a request arriving and the first token leaving? . . . 2
- 1.2 Q2. Stream over SSE, handle disconnects, and cancel in-flight generation . . . 2
- 1.3 Q3. Size KV cache for a 70B model, 32K context, 100 users; account for GQA and precision . . . 2
- 1.4 Q4. Explain PagedAttention and the problem it solves . . . 3
- 1.5 Q5. Why is FlashAttention faster when the math is the same? . . . 3
- 1.6 Q6. Design continuous batching under latency deadlines; why it beats static batching 3
- 1.7 Q7. A 100K-token prompt stalls short chats: fix TTFT with chunked prefill and scheduling 3
- 1.8 Q8. Design prefix caching and cache-aware routing across GPU replicas . . . 4
- 1.9 Q9. Cut inter-token latency without tanking throughput . . . 4
- 1.10 Q10. Speculative decoding: when it helps and when it hurts . . . 4

2 Precision, GPUs, and kernel optimization **4**
- 2.1 Q11. FP8 vs INT8 vs INT4 vs NVFP4: pick one and measure quality loss . . . 4
- 2.2 Q12. Pick GPUs: H100 vs H200 vs B200 vs L40S . . . 5
- 2.3 Q13. Use Nsight to classify a slow model: compute, bandwidth, or launch overhead . . 5
- 2.4 Q14. GPU sits at 30% utilization under load: debug it . . . 6
- 2.5 Q15. Fused CUDA/Triton RMSNorm kernel and benchmark against PyTorch . . . 6

3 Observability and reliability **6**
- 3.1 Q16. Design observability: TTFT, TPOT, queue depth, KV utilization, prefix hit rate, cost/request . . . 6
- 3.2 Q17. p99 is 10x p50: diagnose it . . . 7
- 3.3 Q18. vLLM vs TensorRT-LLM vs SGLang: benchmark real traffic and choose . . . 7
- 3.4 Q19. Safely roll out a new model or quantization, with rollback . . . 7
- 3.5 Q20. Autoscale GPUs with slow cold starts . . . 7
- 3.6 Q21. Multi-tenant serving: fair scheduling, admission control, priority queues . . . 8

4 Distributed inference and cost **8**
- 4.1 Q22. Disaggregate prefill and decode: why, how transfer KV, and what does it cost? . 8
- 4.2 Q23. Serve a large MoE: tensor vs pipeline vs expert parallelism and all-to-all . . . 8
- 4.3 Q24. Reduce cost per million tokens by 70% without hurting quality . . . 9
- 4.4 Q25. Design ChatGPT-style inference for one million daily users: peak QPS, GPU capacity and cost . . . 9

5 Practical checklist **9**

How to answer systems-design questions

For every open-ended question: clarify the workload and SLOs; state assumptions; establish a baseline; estimate the bottleneck; explain the design; measure quality, latency, throughput, reliability, and cost; and describe failure handling. Numbers below are illustrative engineering estimates, not universal benchmark claims.

1 LLM inference and serving

1.1 Q1. What happens between a request arriving and the first token leaving?

Hinglish: Request API gateway par aati hai; authentication/validation hota hai; request queue aur scheduler se guzarti hai; prompt tokenize hota hai; GPU par prefill prompt process karke KV cache banata hai; final prompt position ke logits se first output token select hota hai; phir server token ko stream karta hai. Client tak pahunchne ka end-to-end time TTFT hai. Queueing, preprocessing, prefill, sampling, serialization, network aur proxy buffering ko separately measure karo.

English answer: "I would trace the request from admission and tokenization through scheduling, prefill, first-token sampling, and network delivery. I would break TTFT into queueing, preprocessing, prefill, and delivery components, then optimize the dominant component rather than assuming the GPU is always the bottleneck."

1.2 Q2. Stream over SSE, handle disconnects, and cancel in-flight generation

Hinglish: HTTP response ko Server-Sent Events (SSE) ke through 'text/event-stream' format mein stream karo. Har event mein token/text delta, request ID aur optional usage metadata bhejo; completion par '[DONE]' ya explicit finish event bhejo. Headers mein 'Content-Type: text/event-stream', 'Cache-Control: no-cache', aur proxy buffering disable karne ki configuration rakho. Heartbeats idle connection ko alive rakh sakte hain. Client disconnect ko cancellation signal banao: request context cancel ho, scheduler queued request hata de, aur running decode sequence ko next safe scheduling point par stop karke KV blocks release kare. Cancellation ko best-effort maana jaana chahiye; already launched GPU work turant interrupt na bhi ho. Retry policy idempotency/request IDs ko dhyan mein rakhe.

English answer: "I would stream incremental output as SSE events, send heartbeats when needed, disable intermediary buffering, and include request IDs and explicit completion/error events. A disconnect would cancel the request context, remove queued work, stop future decode steps, and release its KV-cache allocation. I would test cancellation latency and ensure cleanup happens even on errors."

1.3 Q3. Size KV cache for a 70B model, 32K context, 100 users; account for GQA and precision

For a decoder-only transformer, approximate KV bytes per token:

$$M_{\mathrm{KV/token}} = 2\, L\, H_{\mathrm{KV}}\, D_{\mathrm{head}}\, B$$

where 2 represents Key and Value, L is layer count, H_{KV} is the number of KV heads, D_{head} is head dimension, and B is bytes per element. Use the actual model config: parameter count alone does not determine KV-cache size.

Illustrative 70B GQA assumptions: $L = 80$, $H_{\mathrm{KV}} = 8$, $D_{\mathrm{head}} = 128$. Per token in BF16/FP16 ($B = 2$): $2 \cdot 80 \cdot 8 \cdot 128 \cdot 2 = 327{,}680$ bytes, or 320 KiB/token. At 32,768 tokens this is 10 GiB per sequence. For 100 simultaneous sequences at full 32K context, it is about 1,000 GiB = 0.98 TiB, before block rounding, temporary buffers, metadata, fragmentation, and safety headroom. FP8 KV (1 byte/element) approximately halves this to 0.49 TiB, subject to supported scaling/accuracy constraints. Without GQA, e.g. 64 KV heads under otherwise identical assumptions, the cache would be 8 times larger: roughly 7.8 TiB for 100 full contexts. Real systems may use fewer active tokens than the maximum, so size for the concurrency/context distribution and reserve capacity for weights and runtime memory.

English answer: "I would use the model's layer count, KV-head count, head dimension, cache dtype, and active sequence lengths. Under an illustrative 80-layer, 8-KV-head, 128-dimension GQA configuration, BF16 cache is about 320 KiB per token, or 10 GiB per 32K sequence and about 0.98 TiB for 100 such sequences. I would add fragmentation and runtime headroom, then validate against measured allocator usage."

1.4 Q4. Explain PagedAttention and the problem it solves

Hinglish: Traditional implementations may reserve contiguous KV memory per sequence, wasting space because outputs have unknown lengths and requests finish at different times. PagedAttention splits each sequence's KV cache into fixed-size blocks and maps logical token blocks to physical GPU blocks through a block table. Blocks can be allocated on demand, reducing fragmentation and enabling safe sharing of prefix blocks when supported. It improves effective KV utilization and makes continuous batching easier; it does not remove the fundamental KV bytes required by tokens.

English answer: "PagedAttention manages KV cache in fixed-size blocks mapped through block tables, rather than requiring one large contiguous allocation per sequence. It reduces fragmentation, supports incremental allocation and block reuse, and increases the number of active sequences that fit in memory. The trade-offs include block-table management and some block-level waste."

1.5 Q5. Why is FlashAttention faster when the math is the same?

Hinglish: Attention ka mathematical result same ho sakta hai, lekin naive attention poora $N \times N$ score matrix HBM/GPU memory mein materialize karta hai. FlashAttention tiles Q, K, V, SRAM/shared memory mein intermediate calculations karta hai, aur online softmax se output incrementally compute karta hai. Isse HBM reads/writes aur intermediate memory traffic kam hota hai. It is an IO-aware algorithm, not a different attention objective. Exact speedup sequence length, hardware, dtype, shapes and kernel implementation par depend karta hai.

English answer: "FlashAttention speeds up the same attention computation by reducing data movement. It tiles the computation, keeps intermediates in fast on-chip memory, and uses an online softmax so the full attention matrix need not be written to high-bandwidth memory. The gain comes from IO efficiency and fused kernels, not from changing the mathematical result."

1.6 Q6. Design continuous batching under latency deadlines; why it beats static batching

Hinglish: Static batching requests ko fixed batch ke complete hone tak wait kara sakti hai, aur different output lengths ki wajah se finished sequences slots waste karti hain. Continuous/in-flight batching har decode step par finished requests remove aur waiting requests add karti hai. Scheduler ko TTFT/TPOT SLO, request deadline, prompt length, expected output length, priority aur KV budget dekhna chahiye. Prefill aur decode ke compute/memory needs alag hote hain, isliye token-budgeted scheduling aur fairness use karo. Overload mein admission control, bounded queues, rejection ya graceful degradation zaroori hai.

English answer: "I would use iteration-level scheduling: add requests when capacity permits and remove completed or cancelled sequences after each decode iteration. Unlike static batching, this avoids waiting for the longest request and keeps GPU slots productive. I would enforce per-iteration token/KV budgets, prioritize deadlines without starving normal traffic, and measure TTFT, TPOT, throughput, and deadline-miss rate."

1.7 Q7. A 100K-token prompt stalls short chats: fix TTFT with chunked prefill and scheduling

Hinglish: Long prompt ka prefill ek giant workload ban kar short requests ko GPU se block kar sakta hai. Chunked prefill long prompt ko smaller token chunks mein split karta hai. Scheduler decode iterations aur short/urgent prefill chunks ko interleave karta hai, while keeping a maximum prefill token budget per iteration. Chunk size too small hua to kernel-launch overhead badhega; too large hua

to decode latency suffer karegi. Admission control, max context, prompt truncation/summarization and priority lanes bhi consider karo. Chunking total prefill FLOPs magically reduce nahi karta; it improves responsiveness/fairness.

English answer: "I would split long prompts into bounded prefill chunks and schedule those chunks alongside decode work, using token budgets and latency-aware priorities. I would tune chunk size against both TTFT and TPOT, and add admission limits for extreme prompts. This reduces head-of-line blocking but does not eliminate the underlying compute cost of a 100K-token prompt."

1.8 Q8. Design prefix caching and cache-aware routing across GPU replicas

Hinglish: System prompt ya repeated document prefix ke token IDs plus model/version/adapter/position/attention settings se deterministic cache key banao. Prefix KV blocks ko hash-based trie/radix tree ya block hashes se index karo. Cache hit tabhi valid hai jab exact prefix and all computation-affecting settings match hon. Refcounts protect in-use blocks; LRU/LFU or size-aware eviction removes cold blocks. Router ko replica-level cache-hit estimates, free KV memory, queue depth and load balance karna chahiye. Sticky routing helps cache reuse, but can overload one replica; use bounded affinity and fallback routing. Tenant isolation and data privacy must prevent cross-tenant leakage.

English answer: "I would key cached prefix blocks by token sequence and every model/configuration field that affects their KV values, then manage them with reference counts and a memory-aware eviction policy. A router would balance cache locality against queue depth and free KV capacity. I would enforce tenant isolation, version invalidation, and fallback behavior when a cache is unavailable."

1.9 Q9. Cut inter-token latency without tanking throughput

Hinglish: TPOT (time per output token) reduce karne ke liye pehle decode path profile karo. Decode often memory-bandwidth-bound hota hai because each step streams weights and reads/writes KV. Batch size bahut badhane se throughput improve ho sakta hai but individual TPOT and p99 degrade ho sakte hain. Use SLO-aware continuous batching, optimized attention/KV kernels, GQA/MQA, supported quantization, speculative decoding where useful, and avoid CPU synchronization or unnecessary launch overhead. Throughput-latency Pareto curve measure karo; one metric optimize karke doosra ignore mat karo.

English answer: "I would profile per-token decode latency, then tune batch/token budgets, KV access, kernels, precision, and scheduling against a TPOT SLO. I would evaluate speculative decoding only if draft overhead is offset by accepted tokens. Every change would be compared on the throughput-versus-p50/p95/p99 latency curve, not throughput alone."

1.10 Q10. Speculative decoding: when it helps and when it hurts

Hinglish: Small draft model multiple candidate tokens propose karta hai; target model parallel verification karta hai and accepts a prefix according to a correctness-preserving sampling algorithm. It helps when draft tokens are accepted at a high rate, target verification is efficient, and target decoding is the bottleneck. It hurts when acceptance is low, draft model adds expensive compute/memory, batch sizes are already large, or verification increases contention. Measure acceptance length, tokens/sec, TTFT/TPOT, and quality/equivalence to the target sampling distribution.

English answer: "Speculative decoding trades cheap draft-model work for fewer sequential target-model steps. It helps when acceptance rates are high and verification is efficient, but can lose when draft overhead or rejection rates dominate. I would benchmark accepted tokens per draft step, end-to-end latency, throughput, and distributional correctness."

2 Precision, GPUs, and kernel optimization

2.1 Q11. FP8 vs INT8 vs INT4 vs NVFP4: pick one and measure quality loss

Hinglish: FP8 uses floating-point representation with exponent/mantissa; INT8/INT4 use quantized integers and scales/zero-points or group-wise scales; INT4 saves more weight memory but can intro-

duce more error. NVFP4 is NVIDIA's 4-bit floating-point format with specific scaling support in compatible Blackwell-era hardware/software. Format alone doesn't determine accuracy: calibration, per-channel/group scaling, activation/KV precision, outlier handling, kernels and model architecture matter. Practical starting point: FP8 for strong quality/performance where hardware supports it; evaluate INT4/NVFP4 if memory or throughput pressure demands it.

Measure loss with a fixed evaluation set: task accuracy, perplexity (where appropriate), exact-match/quality metrics, long-context tests, safety tests, and representative prompts. Compare same decoding settings and report confidence intervals or bootstrap variation. Also measure tokens/s, TTFT, TPOT, memory and cost. Use a quality non-inferiority threshold defined before testing; don't claim zero loss from a small sample.

English answer: "I would start with FP8 when supported because it often offers a practical quality-memory-throughput balance, then evaluate lower-bit formats if capacity requires them. I would compare the quantized model with a full-precision baseline on representative offline and online evaluations, set a pre-agreed quality-loss bound, and measure latency, throughput, memory, and cost together."

2.2 Q12. Pick GPUs: H100 vs H200 vs B200 vs L40S

Hinglish: Choice workload and deployment constraints par depend karta hai, not just peak FLOPs. H100 is a strong Hopper training/inference accelerator; H200 retains Hopper architecture but adds much larger/faster HBM capacity/bandwidth, often useful for large models and long context. B200 is Blackwell-generation hardware with newer compute/precision capabilities and very high memory bandwidth/capacity, but platform cost, availability, power, software maturity and cluster integration matter. L40S is a lower-cost, air-cooled inference/graphics-oriented GPU with much less memory bandwidth/capacity than top datacenter accelerators; it may fit smaller models, quantized inference or latency-insensitive workloads but is not a drop-in equivalent for large 70B long-context serving.

Decision process: model weight memory + KV cache + activation/workspace; required precision; context and concurrency; target TTFT/TPOT and throughput; interconnect and multi-GPU topology; power/cooling; hourly cost and utilization. Benchmark the exact model/engine at production batch and sequence distributions.

English answer: "I would choose based on memory capacity and bandwidth, supported precision, interconnect, SLOs, and total cost per useful token. H200 is attractive when Hopper compatibility and memory capacity matter; B200 when its newer compute/memory capabilities justify platform cost; H100 as a mature high-end baseline; and L40S for smaller or cost-sensitive workloads. I would validate the choice with workload-specific benchmarks."

2.3 Q13. Use Nsight to classify a slow model: compute, bandwidth, or launch overhead

Hinglish: First use Nsight Systems to see timeline: CPU gaps, launch gaps, synchronization, copies, GPU kernels and overlap. Then use Nsight Compute on key kernels: achieved occupancy, SM utilization, tensor-core activity, memory throughput, roofline position, stalls and kernel duration. Compute-bound: near compute roofline and high math-pipeline use. Memory-bound: high bandwidth or long memory-dependency stalls relative to compute. Launch-bound: many tiny kernels and CPU/GPU gaps, common at small batch/short sequence. Avoid guessing from GPU utilization alone. Profile representative shapes and warmup steady state.

English answer: "I would use Nsight Systems first to locate end-to-end gaps and synchronization, then Nsight Compute to inspect dominant kernels against compute and memory rooflines. High math utilization suggests compute limits; high bandwidth or memory stalls suggests data movement; many short kernels and launch gaps suggests launch overhead. I would optimize the measured bottleneck and re-profile."

2.4 Q14. GPU sits at 30% utilization under load: debug it

Hinglish: 30% utilization itself is not enough to diagnose. Check request arrival rate and queue depth; CPU tokenization/preprocessing; data copies/PCIe; kernel launch gaps; batch size; synchronization; model parallel communication; uneven rank/experts; memory pressure and KV allocation; throttling/power; and whether workload is decode-bound with low per-step parallelism. Use traces and per-GPU/per-rank metrics. Increase batching only if SLOs allow; add continuous batching and remove CPU bottlenecks. Check whether one GPU is the straggler while average utilization hides it.

English answer: "I would correlate GPU metrics with request rate, queueing, CPU utilization, kernel timelines, memory, communication, and per-rank imbalance. Low utilization can mean insufficient batching, CPU/launch bottlenecks, synchronization, or a workload that does not saturate the GPU. I would use traces to identify the cause before changing batch size or hardware."

2.5 Q15. Fused CUDA/Triton RMSNorm kernel and benchmark against PyTorch

RMSNorm computes $y_i = x_i \cdot \text{rsqrt}(\frac{1}{D}\sum_j x_j^2 + \epsilon) \cdot w_i$, usually accumulating the sum in FP32 and applying the learned weight w. A fused row-wise kernel loads one row, reduces squared values, computes inverse RMS, applies scale and writes output. For production, handle non-power-of-two dimensions, masks, dtype/accumulation rules, alignment, strides and numerical tolerance.

Triton sketch (illustrative):

```
import torch
import triton as tr
import triton.language as tl

@tr.jit
def rms_kernel(X, W, Y, D: tl.constexpr,
               EPS: tl.constexpr, BLOCK: tl.constexpr):
    row = tl.program_id(0)
    col = tl.arange(0, BLOCK)
    x = tl.load(X + row * D + col, col < D, 0).to(tl.float32)
    w = tl.load(W + col, col < D, 0).to(tl.float32)
    mean_sq = tl.sum(x * x, axis=0) / D
    inv = tl.rsqrt(mean_sq + EPS)
    y = x * inv * w
    tl.store(Y + row * D + col, y, col < D)
```

Launch with 'BLOCK=tr.next$_p ower_o f_2(D)$'$and one program per row; a production kernel needs appropriate launch co$
$appropriate tolerances. Fusing helps by reducing intermediate reads/writes and launches; a custom kernel is not guar$

English answer: "I would fuse square, reduction, reciprocal-square-root, scale, and store into one row-wise kernel, accumulating in FP32. I would verify numerical agreement and edge cases, then benchmark warm steady-state latency across realistic shapes against PyTorch with correct synchronization. I would keep the custom kernel only if it improves the relevant workload."

3 Observability and reliability

3.1 Q16. Design observability: TTFT, TPOT, queue depth, KV utilization, prefix hit rate, cost/request

Hinglish: Request-level traces should carry request ID, model/version, tenant, prompt/output token counts, queue time, prefill/decode timings, finish/cancel reason, cache-hit status and GPU/replica. Metrics include TTFT and TPOT p50/p95/p99; end-to-end latency; input/output tokens per second; queue depth and age; active sequences; KV blocks used/free; OOM/preemption; prefix-cache hit rate and tokens reused; error/cancellation rates; GPU power/utilization; and cost per request or million tokens. Break down by model, region, priority and prompt/output length, while controlling cardinality and protecting prompt/user data. Alerts should track SLO burn rates, not every noisy metric.

English answer: "I would instrument request traces and expose latency percentiles, queue age, token throughput, active sequences, KV-cache pressure, cache reuse, errors, cancellations, and cost. Metrics would be segmented by model and workload class, with privacy-aware logging and SLO-based alerts. I would connect infrastructure metrics to user-visible latency and cost per useful token."

3.2 Q17. p99 is 10x p50: diagnose it

Hinglish: Percentiles ko prompt length, output length, tenant, region, model version, batch, priority and cache-hit status ke hisaab se slice karo. Common causes: queueing bursts, giant prefills blocking decode, KV pressure/evictions, preemption, GPU stragglers, network/proxy buffering, retries, cold starts, noisy neighbours, lock contention and rare slow kernels. Distributed tracing se p99 requests ke critical path ko reconstruct karo. Arrival-rate spikes and queue age correlate karo; p50 optimize karna enough nahi. Fix root cause with admission control, chunked prefill, fairness, capacity headroom and cache-aware routing, then validate p99 and throughput together.

English answer: "I would trace the slowest requests and segment tail latency by prompt/output size, queue time, cache state, replica, tenant, and model version. I would look for queueing bursts, long-prefill head-of-line blocking, memory pressure, stragglers, retries, cold starts, and network buffering. I would validate the suspected cause with a controlled change and monitor p99, deadline misses, and throughput."

3.3 Q18. vLLM vs TensorRT-LLM vs SGLang: benchmark real traffic and choose

Hinglish: Product name se winner decide mat karo. vLLM is a general-purpose serving engine known for PagedAttention and continuous batching; TensorRT-LLM provides NVIDIA-optimized inference paths and may be attractive when the target stack is NVIDIA and engine/build tuning is acceptable; SGLang emphasizes efficient structured generation and radix/prefix caching workflows. Exact capabilities evolve by version. Replay a privacy-safe representative traffic trace: prompt/output length distributions, concurrency, arrival bursts, structured outputs, cancellation, prefix reuse and tenant priorities. Keep hardware, model weights, precision, sampling, max context and SLOs comparable. Measure TTFT/TPOT p50/p95/p99, tokens/s, quality, memory, failure rate, operational effort and cost.

English answer: "I would benchmark all viable engines on the same hardware and model using a production-representative trace, rather than choosing by headline benchmarks. I would compare latency percentiles, throughput, cache reuse, memory, correctness, reliability, and operational complexity. I would select the engine that meets SLOs at the lowest sustainable total cost and re-test after major version changes."

3.4 Q19. Safely roll out a new model or quantization, with rollback

Hinglish: First offline eval: quality, safety, long context, regression suite, throughput and memory. Then shadow traffic (no user-visible effect), canary on a small percentage, gradually ramp by cohort/region, and compare control/candidate on SLOs and quality proxies. Use immutable model/version configs and reproducible routing. Define rollback triggers before rollout: quality/safety regression, error rate, TTFT/TPOT, OOMs, or cost increase. Rollback should be one-step traffic reroute; keep previous artifacts warm and compatible. For quality that needs human labels, use delayed review and don't rely only on online latency metrics.

English answer: "I would gate the release with offline quality and safety evaluations, run shadow and canary traffic, then ramp gradually while comparing latency, errors, cost, and quality indicators. I would version all artifacts, define automatic rollback thresholds in advance, and keep the previous model ready for immediate traffic restoration."

3.5 Q20. Autoscale GPUs with slow cold starts

Hinglish: GPU capacity instantly start nahi hoti: provisioning, image pull, weights load, engine compile and warmup can take minutes. Scale on leading indicators—arrival rate, queue age, predicted

token demand, active KV usage and SLO burn—not GPU utilization alone. Maintain warm minimum capacity and a buffer; use scheduled/predictive scaling for known peaks. Scale out early, drain before scale-in, use hysteresis/cooldowns to avoid flapping, and preserve replica diversity. During shortage use admission control, bounded queues, smaller model/fallback only if quality policy permits, or explicit overload responses.

English answer: "I would account for the full cold-start time and scale on queue age, incoming token demand, active KV capacity, and forecasted load. I would maintain warm headroom, use predictive scaling and hysteresis, drain replicas safely before scale-in, and apply admission control during overload. GPU utilization alone is a lagging signal."

3.6 Q21. Multi-tenant serving: fair scheduling, admission control, priority queues

Hinglish: Per-tenant quotas and token budgets define karo: concurrent requests, input/output tokens, GPU time or cost. Priority queues may serve interactive/SLA traffic before batch jobs, but use weighted fair queuing/deficit round robin and aging to prevent starvation. Admission controller estimated prompt KV plus output reserve, current free blocks, queue age, deadline and tenant quota check kare. Limit max context/output and reject or defer overload. Monitor noisy-neighbour effects, per-tenant p99 and fairness. Strict priority without fairness can starve low-priority users indefinitely.

English answer: "I would combine admission control and per-tenant quotas with priority classes and a fair scheduler such as weighted fair queuing. Decisions would consider deadlines, token budgets, estimated KV demand, and current capacity. I would enforce context limits, prevent starvation through aging, and monitor per-tenant latency and resource fairness."

4 Distributed inference and cost

4.1 Q22. Disaggregate prefill and decode: why, how transfer KV, and what does it cost?

Hinglish: Prefill is often compute-heavy and processes many prompt tokens in parallel; decode is sequential and often bandwidth/KV-access sensitive. Separate pools let each phase scale and optimize independently. After prefill, transfer KV blocks to decode workers over fast interconnect/network (e.g. GPU-direct paths where supported), with metadata for layer, block, dtype, positions and request identity; ensure transfer completion before decode consumes it. Cost includes KV bytes transferred, network bandwidth/latency, synchronization, duplicate capacity, routing complexity and failures. If transfer bottleneck or workload small, disaggregation may worsen latency/cost. Compare end-to-end TTFT/TPOT and utilization.

English answer: "I would disaggregate when prefill and decode have sufficiently different bottlenecks to justify separate scaling. The prefill worker transfers its KV state and metadata to a decode worker over a high-bandwidth path, with explicit readiness and failure handling. I would quantify transfer time, network cost, extra capacity, and end-to-end latency before adopting it."

4.2 Q23. Serve a large MoE: tensor vs pipeline vs expert parallelism and all-to-all

Hinglish: Tensor parallelism splits layer matrix operations across GPUs and needs frequent collectives; pipeline parallelism places layer ranges on stages but can introduce bubbles and activation transfers; expert parallelism places different MoE experts on different GPUs and routes tokens to selected experts. MoE compute is sparse per token but all-to-all dispatch/combine can dominate, especially with many small/imbalanced expert batches. Optimize expert placement, topology-aware routing, grouped GEMMs, token capacity and load balance; monitor dropped/overflow tokens if capacity limits exist. Often hybrid TP+EP+PP is chosen based on model size, GPU memory and interconnect topology.

English answer: "I would choose parallelism based on model size, memory, layer structure, and interconnect. Tensor parallelism shards operations, pipeline parallelism shards layers, and expert parallelism distributes MoE experts. For MoE, I would profile all-to-all communication, expert

imbalance, and small GEMMs, then use topology-aware placement and hybrid parallelism where beneficial."

4.3 Q24. Reduce cost per million tokens by 70% without hurting quality

Hinglish: 70% reduction guaranteed nahi; target ko measured levers se attack karo. Establish baseline cost per input/output million tokens and quality/SLO guardrails. Then route easy tasks to smaller models, use prefix/KV caching, batch efficiently, use quantization with quality gates, reduce unnecessary context/output tokens, use speculative decoding only where it wins, improve GPU utilization, choose hardware by workload, and separate prefill/decode only if justified. Optimize prompt templates/RAG to avoid redundant tokens. Evaluate quality by task segment and use canary/A-B tests; never hide cost shifts or quality regressions. Cost metric should include idle capacity, retries, networking, storage and engineering overhead where material.

English answer: "I would first establish a trustworthy cost and quality baseline, then combine model routing, prefix reuse, efficient batching, validated quantization, context/output reduction, and better capacity utilization. I would benchmark each lever independently and in combination, segment quality by task, and roll out only if quality and latency guardrails hold. A 70% reduction is a target to validate, not something to promise in advance."

4.4 Q25. Design ChatGPT-style inference for one million daily users: peak QPS, GPU capacity and cost

Hinglish: User count alone enough nahi. Explicit assumptions define karo. Example: 1M daily active users, each 20 requests/day means 20M requests/day; average QPS = 20,000,000/86,400 ≈ 231. If peak is 5x average, peak ≈ 1,157 requests/s. Suppose average request has 1,000 input and 300 output tokens: peak demand is about 1.16M input tokens/s and 347K output tokens/s. Actual capacity depends heavily on model, context, batching, precision, GPU and SLO; benchmark tokens/s per replica at the target p95/p99. If one replica sustains R output tokens/s and P input tokens/s under mixed traffic, estimate replicas separately against each bottleneck, take the larger requirement, then add N+1/failure and utilization headroom. Cost/month = GPU-hours × hourly price plus networking, storage, CPU, observability and idle reserve. Use regional pools, admission control, autoscaling, caching, rate limits, tenant isolation and safe degradation. Report assumptions and ranges, not fake precision.

Illustrative sizing formula:

$$Q_{\text{avg}} = \frac{U\, r}{86400}, \qquad Q_{\text{peak}} = k Q_{\text{avg}}$$

$$N_{\text{GPU}} \approx \max\left(\frac{\text{peak input tokens/s}}{\text{measured input tokens/s/GPU}}, \frac{\text{peak output tokens/s}}{\text{measured output tokens/s/GPU}}\right) \times (1 + h)$$

where U is daily users, r requests per user per day, k peak/average factor, and h capacity headroom. This is a starting estimate; benchmarked mixed-workload throughput and memory/KV constraints determine the final count.

English answer: "I would start by defining request frequency, token distributions, peak-to-average ratio, model mix, and latency SLOs. Under an illustrative one million users making 20 requests/day, average load is about 231 requests/s; a 5x peak is about 1,157 requests/s. I would translate that into input/output token rates, benchmark the target model and serving engine, size for both compute and KV memory, add failure headroom, and calculate total cost from measured GPU-hours rather than theoretical FLOPs."

5 Practical checklist

- Always distinguish TTFT, TPOT, end-to-end latency, throughput and deadline misses.
- Estimate memory from model configuration and live sequence lengths; weights are not the whole memory budget.

- Profile before optimizing. GPU utilization alone does not identify the bottleneck.
- Benchmark the exact engine, model, precision, traffic distribution and SLO.
- Include cancellation, overload, rollback, fairness, cache isolation and failure handling in system designs.
- State assumptions clearly. Capacity and cost estimates are only as good as the workload assumptions and measured per-GPU throughput.